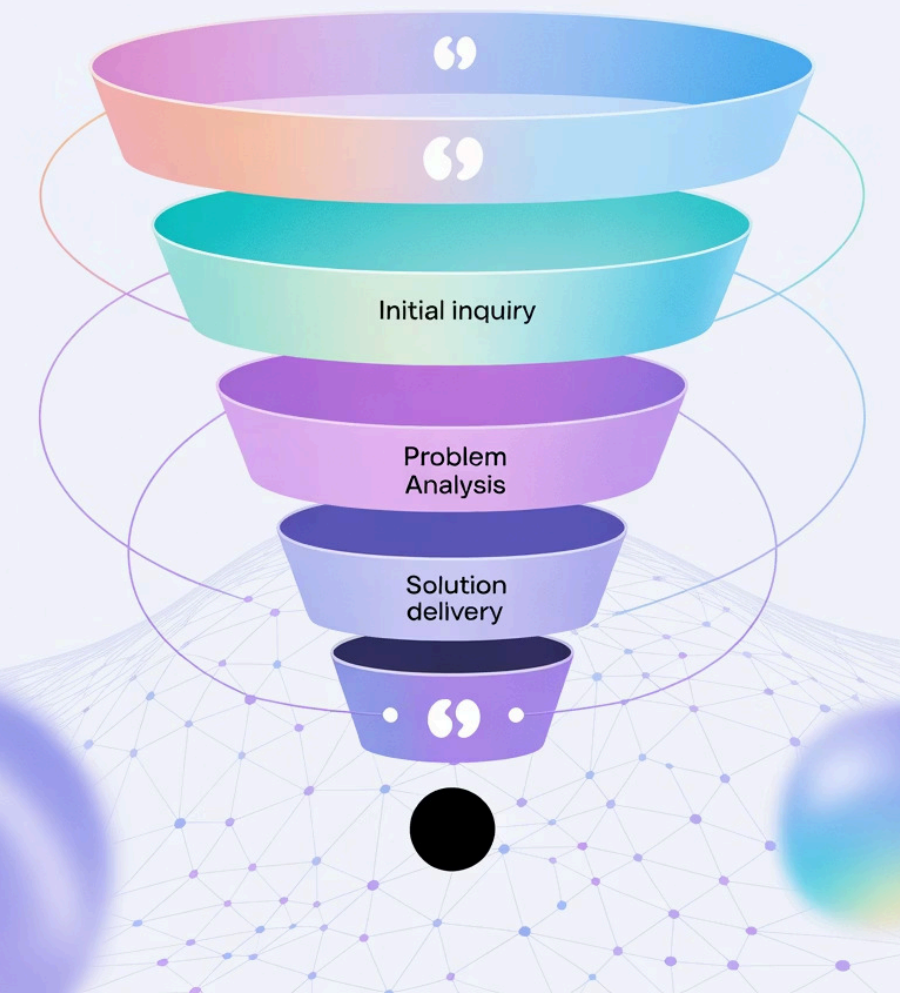


AI

Conversation Funnel



多輪引導漏斗框架：目標導向對話式AI的架構藍圖

對話式AI從簡單的反應式聊天機器人發展為複雜的目標導向代理，這代表了人機交互的典範轉移。本簡報將探討如何建立一個能夠管理結構化多輪對話的架構，以類似傳統銷售漏斗的方式逐步縮小對話範圍，引導用戶向特定且有價值的結果前進。

簡報大綱

01

引導漏斗的概念架構

從銷售漏斗到對話流程的轉換、Dialogue Manager的角色、Slot Filling機制以及「Think, Then Act」循環

02

使用Python和LangGraph的實用框架實現

技術藍圖、DialogueState定義、狀態機構建以及動態Prompt模板的實現

03

進階考量和生產準備

錯誤處理與對話修復、使用RAG增強建議以及長期記憶與個人化



第一部分：引導漏斗的概念架構

本部分為目標導向對話系統建立理論基礎，將以商業為中心的銷售漏斗概念轉換為強健的對話式AI架構。它解構了問題並將其映射到對話系統設計中已建立和新興的模式，為實用且強大的實現奠定基礎。

從銷售漏斗到對話流程：新典範

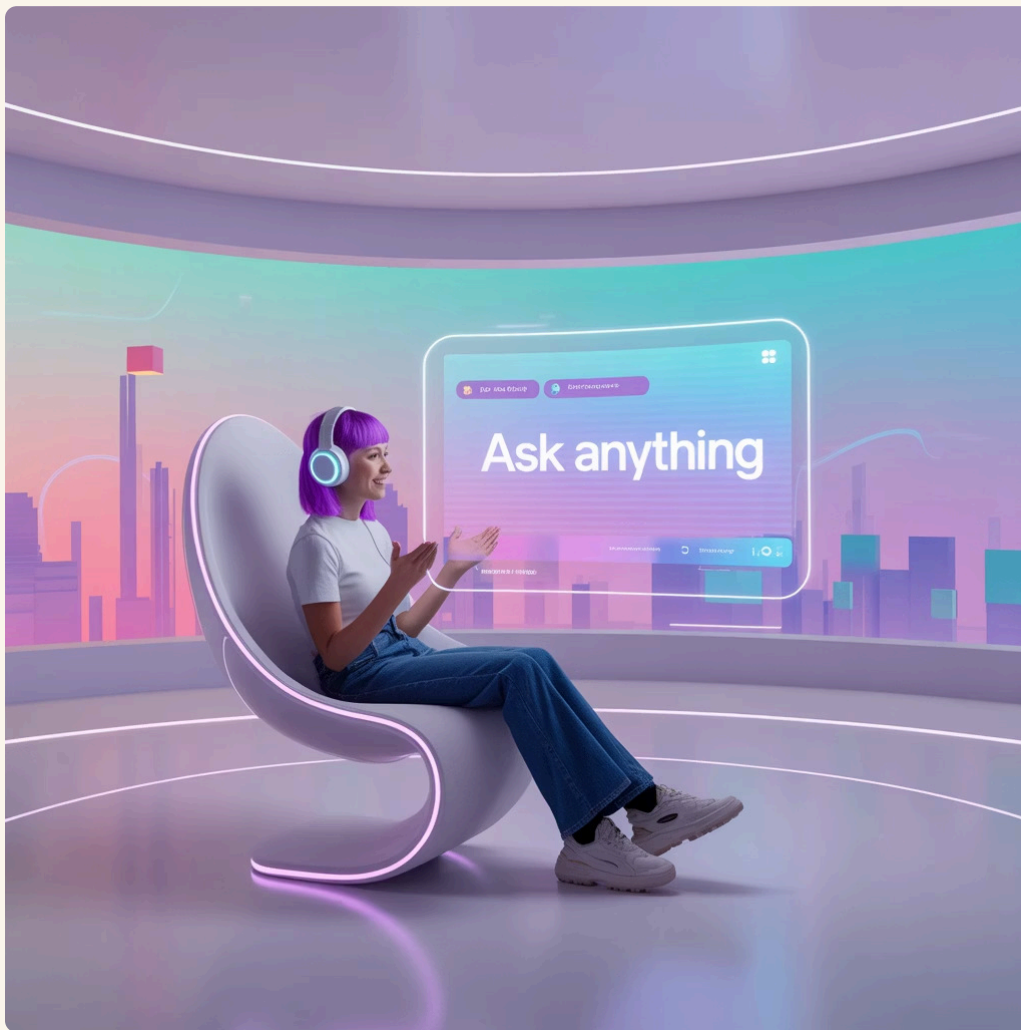


「對話漏斗」的核心概念是將傳統營銷和銷售漏斗直接轉換到自動化對話領域。精心設計的對話代理可以系統性地引導用戶經歷這些階段，從被動的資訊儲存庫轉變為主動的策略指南。

拉取模型 vs. 推送模型

標準RAG應用（拉取模型）

- 等待用戶查詢
- 檢索相關答案
- 用戶完全控制對話方向
- 被動回應用戶需求



引導漏斗框架（推送模型）

- 主動引導對話
- 將對話導向預定目標
- 策略性引導用戶
- 維持持續目標



「引導對話走向真正需求」的商業要求需要「推送模型」，這需要更複雜的架構，包含強健的狀態管理和深思熟慮的多步推理過程。

Dialogue Manager：系統的認知核心

Dialogue State Tracking (DST)

維護對話狀態的結構化表示，包括用戶意圖、已收集的資訊片段以及對話歷史。

對引導漏斗來說，狀態會追蹤用戶處於哪個漏斗階段以及目前為止從他們那裡收集了什麼資訊。

Dialogue Control

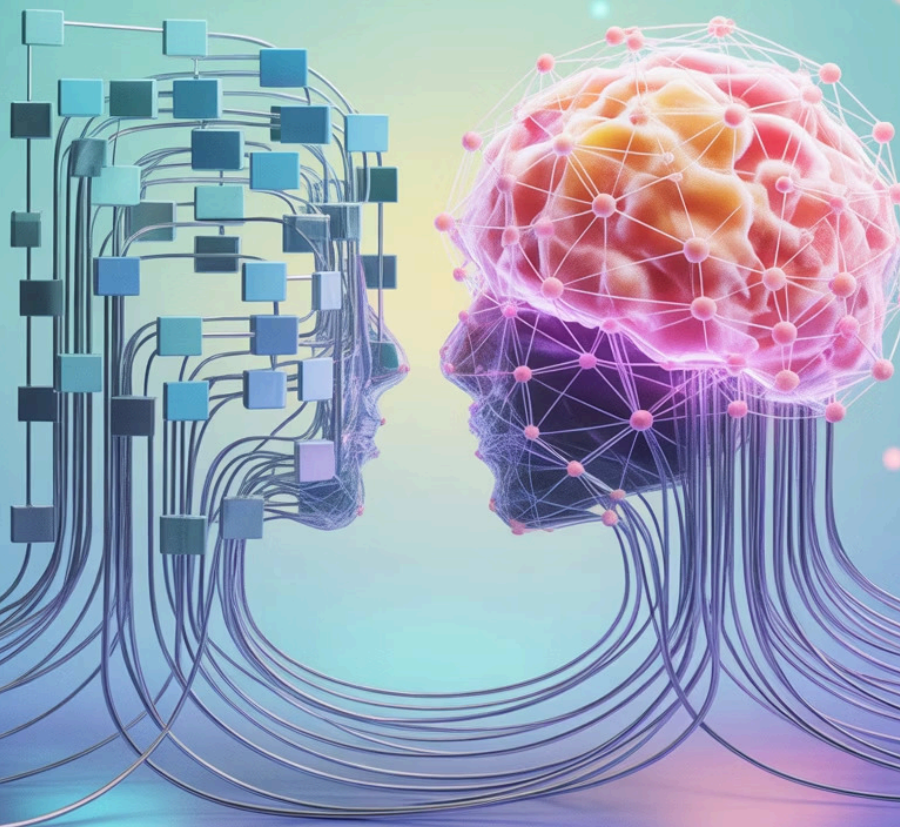
基於當前對話狀態，決定系統應該採取的下一個行動。

在引導漏斗中，對話控制政策至關重要；它體現了推動對話前進的商業邏輯。

Dialogue Manager (DM)是任何先進的任務導向對話系統的核心，負責管理狀態和控制對話流程，作為系統的認知核心。它協調整個互動，接收語意輸入，與外部知識庫介接，追蹤對話進展，並決定系統的下一個行動。

**Rigid
Flowchart**

**Flexible
vs, Neural Network**



LLM作為靈活的機率Dialogue Manager

傳統方法的局限

- 基於規則的系統如Finite-State Machines (FSMs)
- 對話只能沿著預定義的路徑進行
- 可預測但脆弱
- 難以處理人類語言的自然變異性

LLM驅動的DM優勢

- 更靈活的對話流程
- 更好地處理語言變異性
- 但需要結構化以避免偏離軌道
- 需要分離核心功能為不同的LLM驅動步驟

Large Language Models (LLMs)的出現允許新方法：使用LLM本身作為靈活的機率Dialogue Manager。然而，在單一的整體prompt中委託LLM所有DM功能可能不可靠且不透明，特別是當必須嚴格執行複雜商業邏輯時。

Slot Filling：漏斗進展的引擎

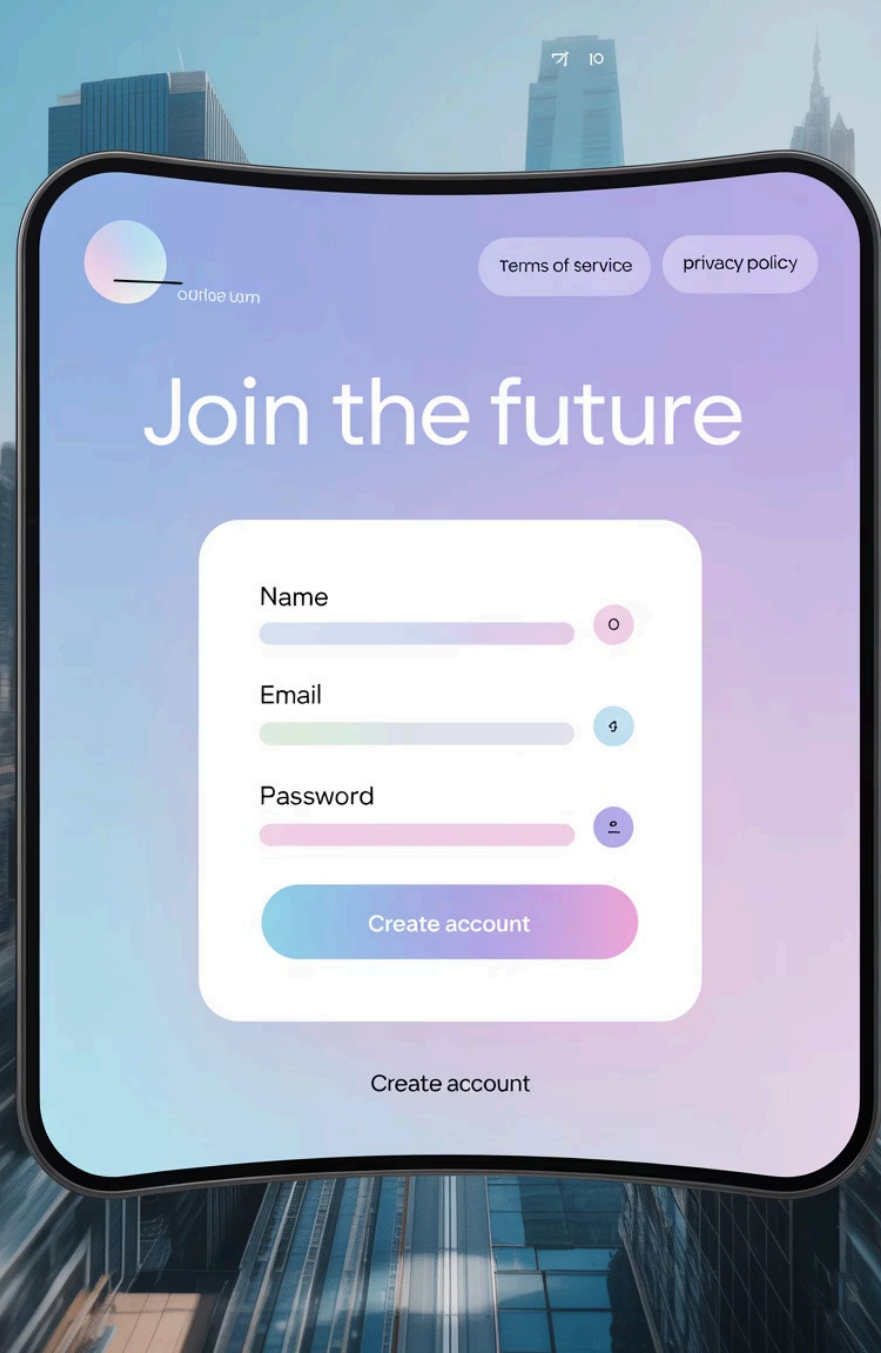
Slot filling是驅動對話漏斗前進的主要機械過程。它是對話式AI中用於從用戶收集特定必需資訊片段以完成任務或意圖的技術。每個填滿的slot代表深入漏斗的一步，將用戶從一般查詢移向特定的可行偏好集。

Slot參數

- Name：slot的唯一識別符
- Entity：slot期望的資訊類型
- Required：指示slot是否為完成任務的必要項
- Is Array：指示slot是否可以接受多個值

Slot填充過程

- 系統查詢其狀態
- 識別下一個必需但目前空白的slot
- 生成針對性問題
- 系統性地縮小可能結果範圍



咖啡銷售Slot架構範例

Slot名稱	數據類型	漏斗中的目的	必需？
taste_profile	List（例如，"fruity","chocolatey", "nutty"）	縮小風味偏好範圍（興趣/評估階段）	是
caffeine_level	String（"regular","decaf", "half-caff"）	確定用戶的咖啡因需求（評估階段）	是
brew_method	String（例如，"espresso", "drip","pour-over"）	將咖啡豆研磨和類型與用戶設備匹配（評估階段）	否
budget_per_bag	Integer	確保建議在用戶價格範圍內（評估階段）	否
experience_level	String（"beginner","intermediate","expert"）	根據推薦咖啡和沖煮建議的複雜性進行調整（個人化）	否

此表格作為DialogueState的單一真實來源，將「推薦咖啡」的抽象目標轉換為將驅動對話的具體數據需求集。

"Think, Then Act"循環：兩步 Prompting的核心



"Think"階段（分析和規劃）

第一個prompt（prompt-lvl1）不是設計來生成面向用戶的文本。其目的是對當前 DialogueState 執行策略分析。

輸出是結構化指令——例如，像{"action":"ELICIT_SLOT", "parameter": "taste_profile"}這樣的JSON物件。

"Act"階段（回應生成）

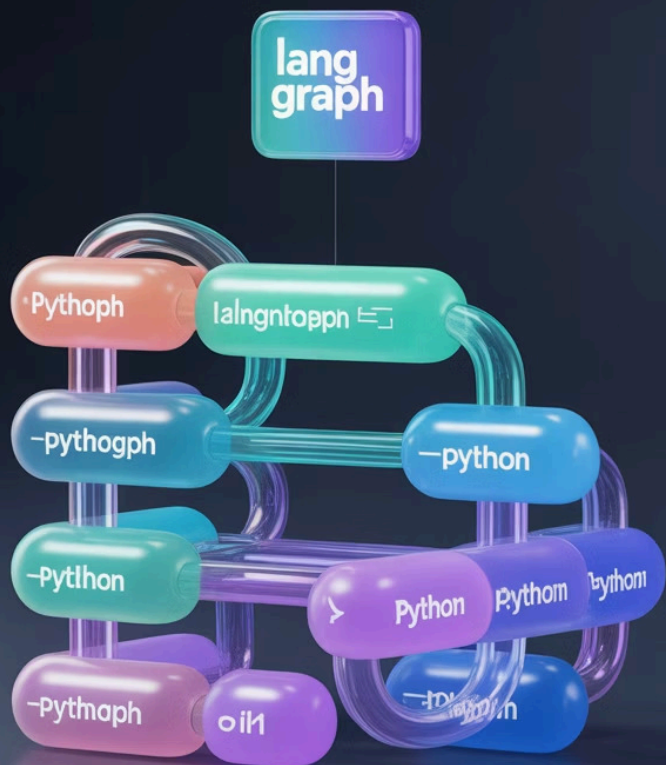
第二個prompt（prompt-lvl2）接收「Think」階段生成的結構化指令作為其主要輸入。

其任務是將該指令轉換為自然、引人入勝且面向用戶的回應。

這種關注點分離不僅僅是實現細節；它是構建強健的目標導向代理的關鍵設計模式。透過在每個步驟約束LLM任務來減輕對話漂移或創意幻想的風險。

第二部分：使用Python和LangGraph的實用框架實現

本部分提供使用Python構建咖啡銷售代理的詳細實用藍圖。它從理論轉向代碼，演示如何構建狀態管理、對話邏輯和動態prompting機制，使「Think, Then Act」循環生效。



技術藍圖：Python、LangChain和LangGraph



Python

作為AI和機器學習開發的事實標準語言，提供豐富的函式庫生態系統、廣泛的社群支援和直接的語法。



LangChain

開源框架，為使用LLMs構建應用提供了全面的工具和抽象集，包括Chat Models、Prompt Templates和Memory元件。



LangGraph

LangChain的擴展，允許開發者透過將它們表示為圖形來構建有狀態的多代理應用，提供StatefulGraphs、Nodes and Edges、Conditional Edging和Cycles功能。

LangChain為與LLMs互動提供基本構件，而LangGraph提供將這些構件編排為我們引導漏斗代理的複雜、有狀態和循環邏輯所需的關鍵控制流程引擎。

定義DialogueState和咖啡架構

DialogueState結構

```
class DialogueState(TypedDict):  
    chat_history: List[BaseMessage]  
    filled_slots: Dict[str, Any]  
    recommendations: List[str]
```

DialogueState是代表代理在對話中任何時點記憶的中央物件。它將在LangGraph應用中的節點間傳遞，確保系統的每個部分都能存取完整的對話語境。

咖啡產品架構

```
def load_coffee_knowledge_base(  
    filepath: str = "coffee_products.csv"  
) -> List:  
    try:  
        df = pd.read_csv(filepath)  
        return df.to_dict(orient='records')  
    except FileNotFoundError:  
        print(f"錯誤：在{filepath}找不到知識庫檔案")  
        return []
```

代理做出相關建議的能力取決於其產品的結構化知識庫。我們使用Pandas函式庫將CSV載入為更易存取的格式，如字典列表。

使用LangGraph構建狀態機

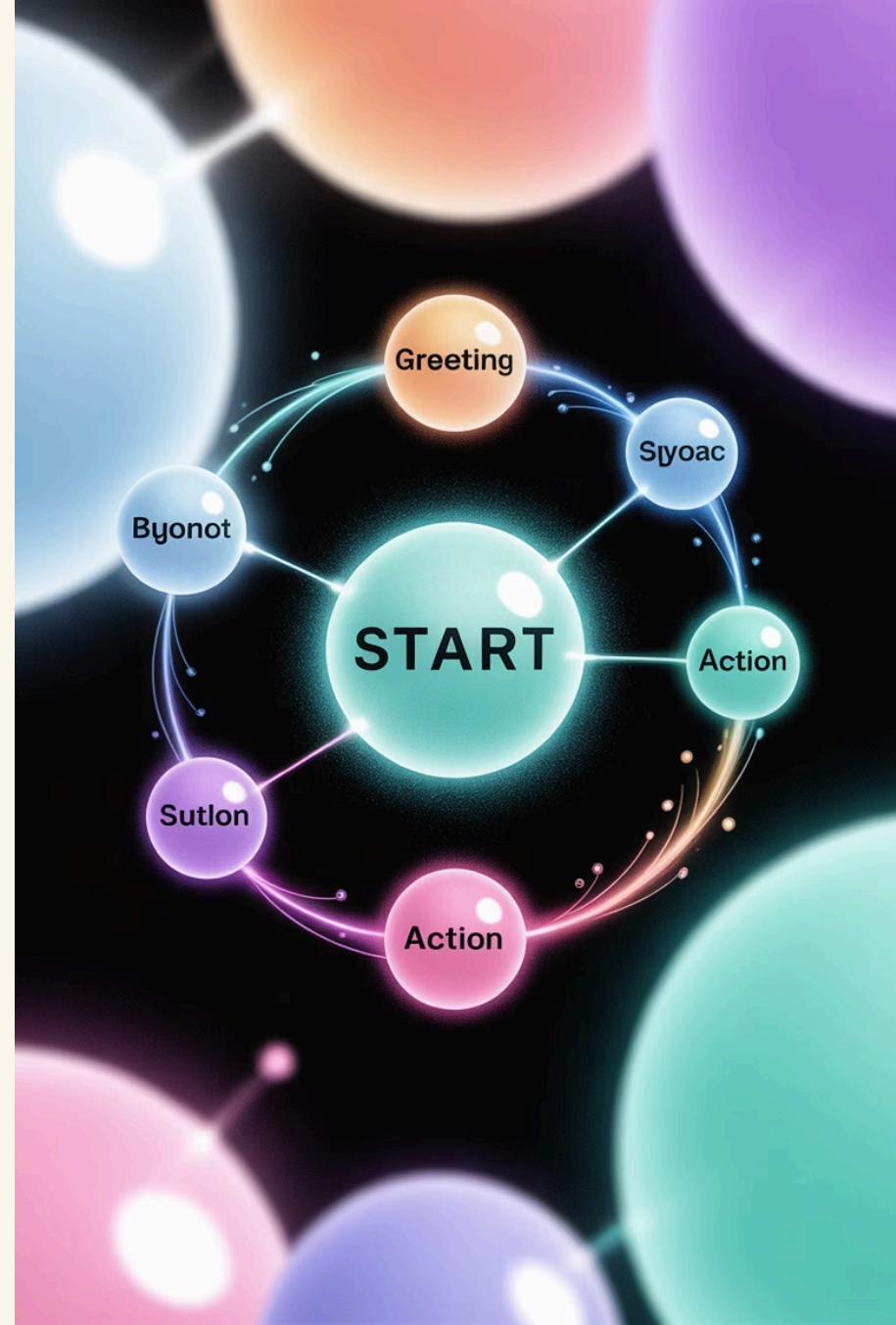
代理被建模為狀態圖，其中節點代表處理步驟（「Think」和「Act」），邊緣指導對話流程。首先，我們初始化StatefulGraph，這個圖綁定到我們的DialogueState物件，該物件將傳遞給每個節點並可被修改。

```
from langgraph.graph import StateGraph, END
```

```
# 初始化圖
```

```
workflow = StateGraph(DialogueState)
```

router是我們圖中最關鍵的節點。它體現我們循環的「Think」部分。它的工作是分析當前DialogueState並決定接下來應該執行哪個「Act」節點。



Router節點 ("Think"步驟)

簡化版Router

```
def route_action(state: DialogueState) -> str:
    """
    這是'Think'步驟。
    它分析狀態並決定下一個行動。
    """
    # 檢查缺失的必需slots
    required_slots = ["taste_profile", "caffeine_level"]
    missing_slots = [slot for slot in required_slots
                      if slot not in state["filled_slots"]]

    if missing_slots:
        # 如果缺少slots，行動是引出資訊
        return "elicit_information_node"
    else:
        # 如果所有必需slots都已填滿，行動是推薦
        return "recommend_product_node"
```

基於LLM的Router

```
# Router Prompt模板
router_prompt_template =
ChatPromptTemplate.from_messages([
    ("system", """"您是咖啡銷售對話的策略決策者。
    當前已填充的slots：{filled_slots}
    必需的slots：taste_profile, caffeine_level
    分析用戶輸入並選擇下一個行動。"""),
    ("human", "用戶訊息：{user_input}\n\n下一個行動是什麼？")
])

# 此prompt會與支援結構化輸出的LLM一起使用
# llm_with_tools = llm.with_structured_output(Route)
# router_chain = router_prompt_template | llm_with_tools
```

為了簡化，初始router使用程式化邏輯。更進階的版本會使用LLM調用來提供更細緻的路由，能夠處理用戶中斷或澄清模糊輸入。

功能節點 ("Act"步驟)

1

elicit_information_node

當router確定缺少必需slot時觸發此節點。它為第一個缺失slot生成澄清問題。

```
def elicit_information_node(state: DialogueState) -> Dict[str, Any]:  
    # 找出第一個缺失的必需slot  
    required_slots = ["taste_profile", "caffeine_level"]  
    missing_slots = [slot for slot in required_slots  
                      if slot not in state["filled_slots"]]  
    slot_to_elicit = missing_slots[0]  
  
    # 使用LLM為slot_to_elicit生成友好問題  
    response_text = f"為了找到完美的咖啡，我需要一點更多資訊。您偏好的{slot_to_elicit.replace('_', ' ')}是什麼？"  
  
    # 更新聊天歷史  
    new_history = state['chat_history'] +  
    [AIMessage(content=response_text)]  
    return {"chat_history": new_history}
```

2

recommend_product_node

當所有必需slots都已填滿時觸發此節點。它篩選產品目錄並使用LLM生成有說服力的建議。

```
def recommend_product_node(state: DialogueState) ->  
    Dict[str, Any]:  
    # 從狀態獲取已填充的slots  
    filled_slots = state["filled_slots"]  
  
    # 根據filled_slots篩選coffee_knowledge_base  
    # ... filtering_logic...  
    potential_recommendations = [] # 範例結果  
  
    # 使用LLM從篩選列表製作有說服力的建議  
    response_text = f"基於您對{filled_slots['taste_profile']}的好  
    評，我推薦試試我們的XYZ咖啡..."  
  
    new_history = state['chat_history'] +  
    [AIMessage(content=response_text)]  
    return {"chat_history": new_history, "recommendations":  
    potential_recommendations}
```



使用條件邊緣組裝圖

```
# 將節點添加到圖
workflow.add_node("handle_user_input_node", handle_user_input_node)
workflow.add_node("elicit_information_node", elicit_information_node)
workflow.add_node("recommend_product_node", recommend_product_node)

# 入口點是處理用戶輸入以提取slots
workflow.set_entry_point("handle_user_input_node")

# 處理輸入後，我們需要決定接下來做什麼
workflow.add_conditional_edges(
    "handle_user_input_node",
    route_action,
    {
        "elicit_information_node": "elicit_information_node",
        "recommend_product_node": "recommend_product_node"
    }
)

# 引出或推薦後，對話結束這一輪，等待下一個用戶輸入
workflow.add_edge('elicit_information_node', END)
workflow.add_edge('recommend_product_node', END)

# 將圖編譯為可執行應用
app = workflow.compile()
```

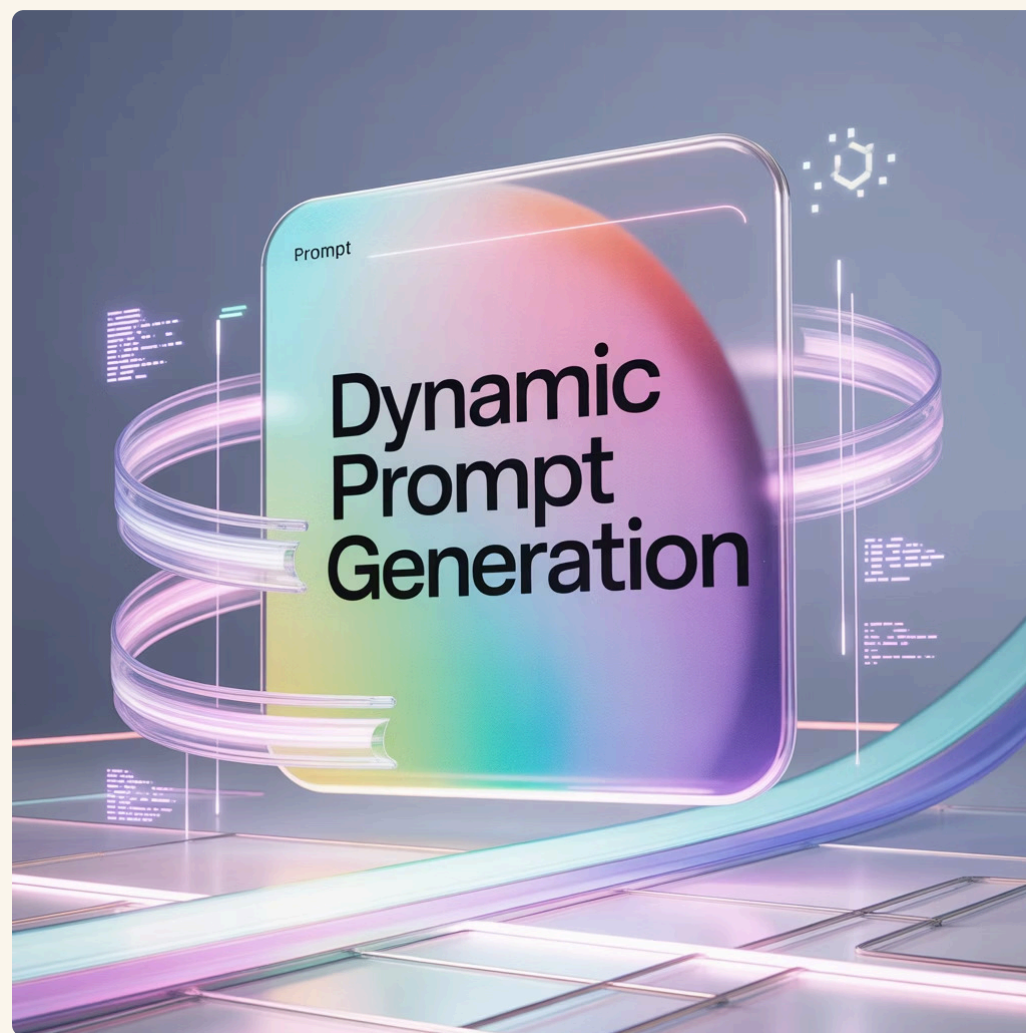
這個圖結構完美實現我們的框架。用戶輸入觸發`handle_user_input_node`，它更新slots。然後，`route_action`函數充當「Think」步驟，決定下一步行動。基於其決定，圖轉換到適當的「Act」節點，它生成回應並結束轉換。

實現動態Prompt模板

引導Prompt ("Act"步驟)

```
# elicitation_prompt_template
elicitation_prompt_template =
ChatPromptTemplate.from_messages([
    ("system", """"您是友好的咖啡專家，正在幫助客戶找到完美的咖啡。
    您需要詢問關於{slot_to_elicit}的問題。
    使用這個範例作為指導：{example_question}
    保持對話自然且引人入勝。"""),
    ("human", "對話歷史：{chat_history}")
])

# 在elicit_information_node中：
# slot_to_elicit = "taste_profile"
# example_question = slot_schema[slot_to_elicit]
["example_question"]
# elicitation_chain = elicitation_prompt_template | llm
```



「Think, Then Act」循環的有效性依賴於在每個階段使用的prompt模板的品質和動態性。這些模板不是靜態的；它們動態地用DialogueState的資訊填充，確保每個LLM調用都精確適應對話的即時語境。

這個兩步prompting機制，其中「Think」prompt的結構化分析輸出（slot_to_elicit）成為「Act」prompt的動態語境輸入，是引導漏斗框架的引擎。它確保代理的對話既策略合理又自然表達。

第三部分：進階考量和生產準備

將對話代理從功能原型移至強健的生產就緒應用需要處理現實世界的複雜性。本部分探索增強引導漏斗框架的進階策略，專注於優雅的錯誤處理、使用先進AI技術改善建議品質，以及啟用真正個人化的長期記憶。



優雅錯誤處理和對話修復

對話修復策略

- 重新表述：嘗試以不同方式提出問題
- 提供選項：從開放式問題轉為多選格式
- 處理中斷：識別和處理用戶想要改變主意或退出流程的意圖

升級給人類代理

- 定義明確的升級觸發器（如連續三次無法理解用戶輸入）
- 檢測強烈負面情緒
- 禮貌地提供連接用戶到人類支援專家
- 確保用戶問題最終得到解決

高品質對話設計的核心原則是系統必須為出錯做好準備。用戶可能提供模糊輸入、離題、表達沮喪或簡單打錯字。生產級代理必須優雅地處理這些情況，而不是失敗或陷入循環。

優雅錯誤處理和對話修復

當對話代理遇到模糊或不完整的用戶輸入時，實施有效的對話修復策略至關重要，以保持對話順暢並避免用戶沮喪。



主動澄清

如果用戶的意圖不明確，主動要求澄清。這可以通過重複理解、提供部分匹配的選項或簡單地要求用戶重新表述來實現。



提供建議或選項

當開放式問題導致用戶困惑時，提供預定義的選項或常見的選擇。這能引導用戶並減少他們思考合適回應的負擔。



回溯和重新引導

如果對話偏離軌道或陷入僵局，代理應能識別並回溯到先前的穩定點，從那裡重新引導用戶回到預期流程。



語境感知糾正

利用對話語境來推斷可能的錯誤或缺失資訊，並自動提出糾正或補充。這減少了用戶需要手動更正的次數。

這些策略旨在讓代理在理解失敗時也能展現出智慧和彈性，從而提升用戶體驗並提高對話完成率。



升級給人類代理

儘管對話式AI系統具備強大的能力，但在某些情境下，將對話無縫升級給人類代理是至關重要的。這不僅確保複雜或敏感問題能得到妥善處理，也極大地提升了用戶體驗和信任度。



識別升級時機

當代理連續無法理解用戶意圖、檢測到用戶沮喪情緒，或用戶明確要求人工協助時，應觸發升級機制。



平滑交接流程

在升級過程中，系統應將完整的對話歷史和已填充的關鍵資訊傳遞給人類代理，避免用戶需要重複敘述。



確保客戶滿意

升級的最終目標是確保用戶問題得到有效解決，並維護正面的客戶關係。這也為AI學習提供寶貴的人工回饋。